

Reporte de la práctica

Temporización no bloqueante — Parte 1: el antipatrón con delay()

1. Datos generales

Materia	Sistemas Programables
Institución	Instituto Tecnológico de Mazatlán (TecNM)
Alumno	Jesús Antonio Rodríguez Peraza — L231000105
Práctica	Práctica 2.1.1 (Parte 1) — Temporización no bloqueante: el antipatrón con delay()
Placa utilizada	Arduino UNO R4 WiFi
Componentes	Arduino UNO R4 WiFi, 3 LEDs, 3 resistencias de 220 Ω , protoboard, cables Dupont
Programa	ParpadeoDelay.ino (Codigo/ParpadeoDelay/)
Repositorio	github.com/Jesus-Jazisenot/Universidad → Sistemas-Programables/Practica-2-Temporizacion-No-Bloqueante-Parte-1-delay

2. Objetivo

Intentar que tres LEDs parpadeen con periodos distintos (500, 1000 y 1500 ms) usando delay() para comprobar, con los propios ojos, que los retardos bloqueantes detienen por completo la ejecución y no permiten que varias tareas con periodos distintos convivan en un solo loop(). Esta parte sirve de contraste para la Parte 2, que resuelve el mismo problema con millis().

3. Diagrama del circuito (conexiones)

LED	Pin Arduino	Resistencia	Periodo pedido
LED 1	8	220 Ω a GND	500 ms
LED 2	9	220 Ω a GND	1000 ms
LED 3	10	220 Ω a GND	1500 ms

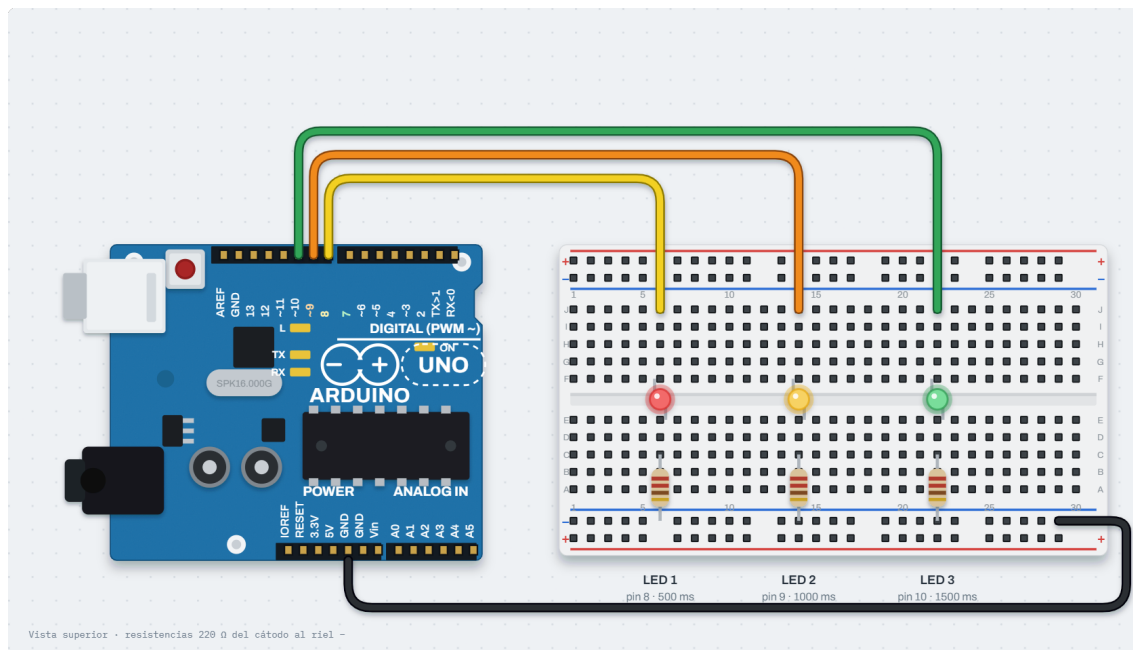


Figura 1. Diagrama de conexiones del circuito.

4. Procedimiento

1. Se armó el circuito en la protoboard: cada LED del pin correspondiente (8, 9 y 10) al ánodo, y el cátodo por una resistencia de $220\ \Omega$ a GND.
2. Se cargó ParpadeoDelay.ino en el UNO R4 WiFi. El programa enciende y apaga cada LED con digitalWrite() y delay(), uno tras otro, con los tiempos pedidos.
3. Se observó el parpadeo y se grabó el video del funcionamiento.
4. Se contó cuántas veces enciende cada LED en un ciclo completo y se comparó contra lo que pedía la práctica (tabla de la sección 5).

5. Comportamiento observado contra el pedido

Lo que se observa en la protoboard es una sola secuencia de 6000 ms en la que cada LED enciende una vez, uno después del otro, y nunca dos al mismo tiempo:

Tiempo (ms)	LED1	LED2	LED3	Instrucción en ejecución
0 – 500	ON	off	off	delay(500)
500 – 1000	off	off	off	delay(500)
1000 – 2000	off	ON	off	delay(1000)
2000 – 3000	off	off	off	delay(1000)
3000 – 4500	off	off	ON	delay(1500)
4500 – 6000	off	off	off	delay(1500)

Comparación entre lo que pedía la práctica y lo que produce el programa con delay():

LED	Periodo pedido	Encendidos esperados en 6 s	Encendidos reales en 6 s
LED 1	500 ms	6	1
LED 2	1000 ms	3	1
LED 3	1500 ms	2	1

** Tiempos medidos con el video del funcionamiento y comparados contra la suma de los delay() del loop(). La práctica no usa el puerto serie, por lo que no hay salida del Monitor Serie.*

6. Observaciones sobre el comportamiento del sistema

- Los LEDs encienden por turnos (rojo, amarillo, verde) y el ciclo completo tarda 6 s, exactamente la suma de los seis delay() del loop().
- En ningún momento hay dos LEDs encendidos a la vez, aunque los periodos pedidos (500, 1000 y 1500 ms) deberían coincidir encendidos en varios instantes.
- El programa compila y "funciona" sin ningún error; el problema solo se ve al comparar el ritmo real contra el pedido. Es un error de diseño, no de sintaxis.
- Durante cada delay() el Arduino no puede reaccionar a nada: si se agregara un botón, la pulsación se perdería hasta que termine la espera en curso.

7. Conclusiones

delay() no significa "espera 500 ms para este LED": significa "detén todo el programa 500 ms". Mientras el procesador está dentro de un delay() no revisa ningún otro LED, ni un botón, ni el puerto serie; por eso los tres LEDs quedaron encadenados en una sola secuencia de 6 s y ninguno cumplió su periodo.

Con retardos bloqueantes es imposible que tareas con periodos distintos convivan en un solo loop(): el tiempo de cada una se suma al de las demás. Se puede maquillar con un paso base y contadores, pero en cuanto aparece una tarea que no es múltiplo de ese paso, o una entrada que deba atenderse al instante, vuelve a fallar.

En un sistema embebido real (sensores que muestrean, comunicación que llega en cualquier momento, botones del usuario) esta forma de programar es un antipatrón: el sistema se vuelve "sordo" durante cada espera. La solución no es acortar los delay() sino no esperar, que es lo que se hace en la Parte 2 con millis().

8. Evidencia del armado

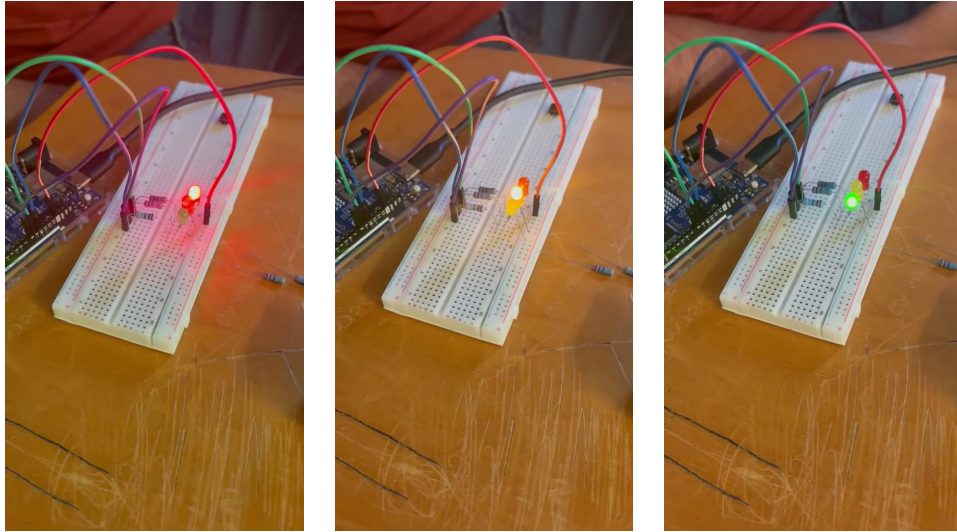


Figura 2. Los LEDs encienden uno después del otro, nunca dos a la vez (cuadros del video).

Video del funcionamiento: <https://www.youtube.com/shorts/sUPkjt看6g>